

Register, end to end

The whole use case is one method. No framework noise, no service-of-services — just orchestration you can read aloud. Everything it needs arrives as an interface; everything it returns is a value the caller is forced to handle.

security/system/registration/Register.java

```
public class Register {
    private final UserRepository userRepository; // ← ports, not classes
    private final CanRegister canRegister; // ← email policy
    private final CreatePasswordHash createPasswordHash; // ← password policy

    // constructor omitted — plain dependency injection

    public RegisterResult execute(Email email, PlaintextPassword password) {
        /*1*/ Decision<Email> emailDecision = canRegister.evaluate(email);
        List<String> emailErrors = emailDecision.errorCodes();

        /*1*/ Outcome<HashedPassword> passwordOutcome = createPasswordHash.create(password);
        List<String> passwordErrors = passwordOutcome.errorCodes();

        /*2*/ return !emailErrors.isEmpty() || !passwordErrors.isEmpty() ?
            new RegisterResult.Invalid(emailErrors, passwordErrors) :
        /*3*/ passwordOutcome.findValue()
            .map(hash -> {
                User userToSave = new User(email, hash);
                /*4*/ User savedUser = userRepository.save(userToSave);
                return new RegisterResult.Valid(savedUser);
            })
            .orElseThrow();
    }
}
```

- 1 **Validate both, independently.** Email and password are checked in full before anything is decided — the user gets every error at once, not one round-trip per mistake.
- 2 **Errors are values, not exceptions.** Failure is a returned `Invalid`, carrying the exact error codes. Nothing is thrown for the expected unhappy path.
- 3 **The happy path can't run on bad data.** Hashing & persistence live behind the success branch — structurally unreachable when validation failed.
- 4 **Persistence is the last step, through a port.** `UserRepository` is an interface; the use case never knows whether it's JPA, in-memory, or a mock.

The supporting cast

The use case stays short because the hard parts are named types that live elsewhere. Each one does a single job and has a single reason to change.

● Decision<T> / Outcome<T>

Sealed three-state results — Allowed, AllowedWithWarning, Rejected. The compiler forces the caller to handle every branch; no result is ever silently dropped.

● Constraints<T>

Replaces the recurring `stream().filter().map(code)` sprawl with one named operation: `decide(candidate)`. Rules become a list you compose, not code you rewrite.

email/policy/CanRegister.java — rules composed, not hard-coded

```
CanRegister.builder()
    .blockingDomains(blocked)           // reject corporate blocklist
    .blockingDisposable(disposable)     // reject throw-away inboxes
    .requiringCompanyEmployee(company)  // only @acme.com may register
```

SAME DOMAIN, DIFFERENT STRICTNESS

A security microservice composes the strict set above. A newsletter service reuses the *identical* Email domain with a looser policy — by changing config, never the rules.

The test is the documentation

jqwik property → runs every input combo → Allure report → living spec

```
@Property
@Label("Invalid when email, password, or both fail validation")
void invalid_when_any_validation_fails(@ForAll boolean emailFails,
                                       @ForAll boolean passwordFails) {
    Assume.that(emailFails || passwordFails);
    // ... arrange the two policies to fail/pass ...
    RegisterResult result = register.execute(email, password);
}
```

- **Property-based, not example-based.** jqwik enumerates the input space — the email-fails, password-fails and both-fail cases are proven, not hand-picked.
- **One assertion proves the safety rule:** on any failure, `save()` is never called.
- **The spec can't drift from the code, because it is the code** — and Allure publishes it as readable documentation.